Assignment

```python
# variable declaration
my_int = 30
my_float = 1.23

my_list = [-1, -2, -3]
my_tuple = (5, 10, 15)
my_set = {3, 5, 7}

my_dict = {
    'key0' : 'value0',
    'key2' : 'value2',
}
```

Access

```python
# accessing and indexing
my_list[0] # -1
my_dict['key0'] # value0
```

Flow Control

```python
# flow control
if condition1:
    execute_code()
elif condition2:
    execute_code()
else:
    execute_code()

match var:
    case value1:
        execute_code()
    case value2:
        execute_code()
    case _:
        default_case()

try:
    execute_code()
except:
    if_error()
```

Loops

```python
# loops
for item in iterable:
    execute_code(item)
    continue # skips code to next iteration
    break # skips code and breaks loop

while condition:
    execute_code()
    update_condition()
```

List & Dict Comprehension

```python
# comprehension
list_comp1 = [expression for item in iterable]
list_comp2 = [expression for item in iterable if condition]
list_comp3 = [expression1 if condition else expression2 for item in iterable]

dict_comp = {key_expression : value_expression for item in iterable}
```

Functions & Classes

```python
# function and class
def function_name(arguments):
    return value

class Class:
    def __init__(self):
        constructor()

    def method(self):
        execute_code()
```

Numpy Basics

```python
np.array() # make array
np.asarray() # cast to array

# array attributes
arr.shape # shape of arr
arr.ndim # number of dimensions
arr.size # total size
arr.dtype # data type
arr.dtype.name # name of data type

# array manipulation
arr.astype(dtype) # convert to type
arr.copy() # copy arr
arr.sort() # sort arr
arr.ravel() # flatten arr
arr.reshape(shape) # reshape arr
arr.T # transpose

np.vstack(arrs) # vertically stack arrs
np.hstack(arrs) # horizontally stack arrs
np.concatenate(arrs, axis=0) # concat arrs along axis
np.append(arr, vals) # add to end
np.insert(arr, index, vals) # insert at index
np.delete(arr, index, axis=0) # delete value at index

# operations accepts axis and keepdims
arr.mean()
arr.min()
arr.max()
arr.std()
np.median(arr)

# numpy math operations
np.add(a, b) # a + b
np.subtract(a, b) # a - b
np.multiply(a, b) # a * b
np.divide(a, b) # a / b

np.log(a)
np.exp(a)
np.pow(a, power)
np.sqrt(a)
np.sin(a)
np.cos(a)
```

Importing

```python
# importing
import package.subpack as alias
from package.subpack import function_class_or_var
```

Pandas Reference

```python
# creation
df = pd.DataFrame({'a':[1,2,3], 'b':[4,5,6]}) # create DataFrame
s = pd.Series([10,20,30]) # create Series

# attributes
df.shape # (rows, cols)
df.index # index labels
df.columns # column labels
df.dtypes # column data types

# inspection
df.head(n) # first n rows
df.info() # summary info (dtypes, non-null)
df.describe() # numeric stats

# selection
df['a'] # select column
df[['a','b']] # select multiple columns
df.loc[0,'a'] # label-based scalar
df.loc[0:1, ['a','b']] # label-based slice
df.iloc[0,1] # position-based scalar
df.iloc[0:2, 0:2] # position-based slice

# add, change, remove columns
df['c'] = df['a'] + df['b'] # new column
df.assign(d=df['a']*2) # return new df with new column
df.assign(new_col=lambda d: d['a']+d['b']) # alt
df.drop(index=[0], columns=['c']) # drop cols or idxs, can be boolean mask

# sort
df.sort_values('b') # sort by column
df.sort_values(['a','b'], ascending=[False, True]) # multi-column sort

# select and rename
df.rename(columns={'a':'A'}) # rename columns
df.filter(['a']) # select columns

# missing data
df.isna() # boolean missing values
df.dropna() # drop missing rows
df.fillna(0) # fill missing with value

# aggregation and groupby
df.groupby('a').sum() # group and aggregate
df.groupby('a').agg(['mean', 'std']) # customized aggregation
df.groupby('a').agg(new_col=('old_col', 'mean'))
df['b'].mean() # column mean
df['b'].sum() # column sum
df['b'].min() # column min
df['b'].max() # column max

# transformation
df['b'].apply(lambda x: x*2) # row-wise apply on Series
df.applymap(lambda x: x*2 if isinstance(x,(int,float)) else x) # elementwise DataFrame

# reshape
df.pivot(index='a', columns='b', values='c') # pivot to wide
df.melt(id_vars=['a'], value_vars=['b','c']) # melt to long

# combine
pd.concat([df, df]) # concat DataFrames
pd.merge(df, df, on='a', how='inner') # merge/join operations
df.join(df.set_index('a'), on='a', rsuffix='_right') # join by index

# conversion
df['a'].to_list() # Series to list
df.to_numpy() # DataFrame values to array
df.to_dict() # to dict

# I/O
pd.read_csv('in.csv', index_col=0) # read in CSV
df.to_csv('out.csv', index=False) # export CSV
```

Keywords / Tropes

Keywords / Tropes	Description
var[...]	Index with integer or slice(array, lists) or accesswith a key value (dicts).
obj.thing	Accessthe attribute or method of an object
self.thing	Within classmethod, accessits own attribute or method
continue	Within a loop, skips rest of code and goes to next iteration
break	Within a loop, skips rest of code and break out of loop
return	Within a function, what value to return
**dict	Unpacks a dict into (key, value) tuple pairs
var:type = value	": type" tells interpreter and reader what type the variable is
def func() -> type	"-> type" tells interpreter and reader what type the function returns
iterable[start:stop]	Slicethe iterable from "start" (inclusive) to before "end" (exclusive)

List Methods

Method	Description
append()	Adds an element at the end of the list
clear()	Removesall the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removesthe element at the specified position
remove()	Removesthe first item with the specified value
reverse()	Reversesthe order of the list
sort()	Sorts the list

Dictionary Methods

Method	Description
clear()	Removesall the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removesthe element with the specified key
popitem()	Removesthe last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary

Built-in Functions

Function	Description
abs()	Returns the absolute value of a number
all()	Returns True if all items in an iterable object are true
any()	Returns True if any item in an iterable object is true
bin()	Returns the binary version of a number
bool()	Returns the boolean value of the specified object
callable()	Returns True if the specified object is callable, otherwise False
chr()	Returns a character from the specified Unicode code.
complex()	Returns a complex number
delattr()	Deletes the specified attribute (property or method) from the specified object
dict()	Returns a dictionary (Array)
dir()	Returns a list of the specified object's properties and methods
divmod()	Returns the quotient and the remainder when argument1 is divided by argument2
enumerate()	Takes a collection (e.g. a tuple) and returns it as an enumerate object
eval()	Evaluates and executes an expression
filter()	Use a filter function to exclude items in an iterable object
float()	Returns a floating point number
format()	Formats a specified value
getattr()	Returns the value of the specified attribute (property or method)
globals()	Returns the current global symbol table as a dictionary
hasattr()	Returns True if the specified object has the specified attribute (property/method)
help()	Executes the built-in help system
hex()	Converts a number into a hexadecimal value
id()	Returns the id of an object
input()	Allowing user input
int()	Returns an integer number
isinstance()	Returns True if a specified object is an instance of a specified object

String Methods

Method	Description
capitalize()	Converts the first character to upper case
casefold()	Converts string into lower case
center()	Returns a centered string
count()	Returns the number of times a specified value occurs in a string
encode()	Returns an encoded version of the string
endswith()	Returns true if the string ends with the specified value
expandtabs()	Sets the tab size of the string
find()	Searches the string for a specified value and returns the position of where it was found
format()	Formats specified values in a string
format_map()	Formats specified values from a dictionary in a string
index()	Searches the string for a specified value and returns the position of where it was found
isalnum()	Returns True if all characters in the string are alphanumeric
isalpha()	Returns True if all characters in the string are in the alphabet
isascii()	Returns True if all characters in the string are ascii characters
isdecimal()	Returns True if all characters in the string are decimals
isdigit()	Returns True if all characters in the string are digits
isidentifier()	Returns True if the string is an identifier
islower()	Returns True if all characters in the string are lower case
isnumeric()	Returns True if all characters in the string are numeric
isprintable()	Returns True if all characters in the string are printable
isspace()	Returns True if all characters in the string are whitespaces
istitle()	Returns True if the string follows the rules of a title
isupper()	Returns True if all characters in the string are upper case
join()	Converts the elements of an iterable into a string
ljust()	Returns a left justified version of the string